**BRAC University**
**Department of Computer Science and Engineering**

Examination: Quiz 01 Semester : Spring26
Duration: 20 Minutes Full Marks: 15

CSE420: Compiler Design

| Name: | ID: | Section:13 |
|---|---|---|

1. A symbol table is a table used by the compiler to store details about identifiers of the source code. The symbol table helps the compiler perform several tasks during different phases of compilation.
    a. Stores Identifier Information: Whenever the compiler finds a new identifier, it stores its information in the symbol table
    b. Detects Duplicate Declarations: If a variable is declared more than once in the same scope, the compiler checks the symbol table to detect the error.
    c. Type Checking: The compiler uses the symbol table to verify that operations use compatible data types.
    d. Scope Management: The symbol table helps track whether a variable is local or global and where it can be used.
    e. Memory Allocation: The compiler uses the symbol table to assign memory addresses to variables.
    f. Helps in Code Generation: During the final stage of compilation, the compiler uses the symbol table to generate machine code using correct memory addresses and variable types.
2. Suppose a situation arises in which the lexical analyzer is unable to proceed because none of the patterns for tokens matches any prefix of the remaining input. The simplest recovery strategy is "panic mode" recovery. We delete successive characters from the remaining input, until the lexical analyzer can find a well-formed token at the beginning of what input is left. This recovery technique may confuse the parser, but in an interactive computing environment it may be quite adequate.

    Panic mode sometimes skips too much input. Because of this, the compiler may lose the correct structure of the program, which can cause it to detect additional errors that are not really present.

1. The analysis part breaks up the source program into constituent pieces and imposes a grammatical structure on them. It then uses this structure to create an intermediate representation of the source program. If the analysis part detects that the source program is either syntactically ill formed or semantically unsound, then it must provide informative messages, so the user can take corrective action. The analysis part also collects information about the source program and stores it in a data structure called a symbol table, which is passed along with the intermediate representation to the synthesis part.

    The synthesis part constructs the desired target program from the intermediate representation and the information in the symbol table. The analysis part is often called the front end of the compiler; the synthesis part is the back end.